

Code Explanation: Phase 5 - Performance Evaluation and Interpretation

Project: NeuroVision (Brain Tumor Classification) **Objective:** To evaluate the trained model's predictive performance using specific metrics and visualizations, directly aligning with the Unit V syllabus requirements.

1. Generating Predictions

Before evaluating the model, we must force it to take a test on the unseen validation data.

- `y_pred_prob = model.predict(X_test)`: We feed the 20% testing images (`X_test`) into the model. The model outputs a probability score (between 0.0 and 1.0) indicating how likely it thinks a tumor is present.
- `y_pred_classes = (y_pred_prob > 0.5).astype(int)`: This applies a threshold. If the probability score is greater than 50% (0.5), it converts the prediction to 1 (Tumor). If it is lower, it converts it to 0 (Normal).

2. Classification Report

This generates a comprehensive text report of our core evaluation metrics.

- `classification_report(...)`: This function from scikit-learn automatically calculates the following metrics for both the 'Tumor' and 'Normal' classes:
 - **Precision**: Out of all the cases the model *predicted* as Tumor, how many were actually Tumors?
 - **Recall (Sensitivity)**: Out of all the *actual* Tumors in the dataset, how many did the model successfully find? In medical AI, high Recall is crucial so patients are not misdiagnosed.
 - **F1-Score**: The harmonic mean of Precision and Recall, providing a single score that balances both metrics.
 - **Accuracy**: The overall percentage of correct predictions across all classes.

3. Confusion Matrix Visualization

A confusion matrix provides a direct, visual breakdown of where the model got confused.

- `cm = confusion_matrix(y_test, y_pred_classes)`: Calculates the raw numbers for True Positives, True Negatives, False Positives, and False Negatives.
- `sns.heatmap(...)`: Uses the Seaborn library to plot these numbers on a color-coded grid (heatmap). `annot=True` ensures the exact numbers are printed inside

the colored boxes, making it very easy to interpret how many healthy scans were misclassified as tumors, and vice versa.

4. Training History Visualization

This plots the model's learning journey across the epochs to visualize convergence and overfitting.

- `plt.subplots(1, 2, ...)`: Creates a 1x2 layout for two separate charts side-by-side.
- `axes[0].plot(history.history['accuracy'] ...)`: Plots the upward curve of the training and validation accuracy over time.
- `axes[1].plot(history.history['loss'] ...)`: Plots the downward curve of the training and validation loss (error) over time. If the training loss keeps dropping but the validation loss starts rising, it is a clear visual indicator of overfitting